

JavaScript

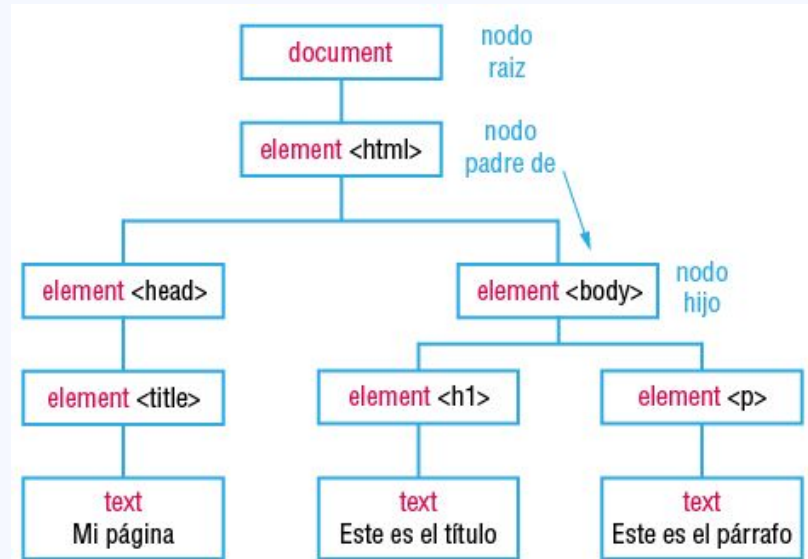
DOM - Eventos - Formulario

Document Object Model (DOM)

DOM (*Document Object Model* o **Modelo de Objetos del Documento**) es una interfaz de programación que representa un documento HTML o XML como una **estructura de árbol**.

En términos sencillos: es el puente que permite a **JavaScript** interactuar, modificar y manipular todo lo que ves en una página web.

```
<html>
  <head>
    <title> Mi página </title>
  </head>
  <body>
    <h1> Este es el título </h1>
    <p> Este es el párrafo </p>
  </body>
</html>
```



Operaciones principales con el DOM

Acción	Método de JavaScript	Ejemplo de uso
Seleccionar	<code>document.querySelector()</code>	Encontrar un botón o un párrafo específico.
Modificar contenido	<code>.textContent</code> , <code>.innerHTML</code>	Cambiar un texto o insertar nuevo código HTML.
cambiar estilos	<code>.style.property</code> , <code>.classList</code>	Modificar el color de fondo o alternar una clase CSS.
Escuchar eventos	<code>.addEventListener()</code>	Ejecutar código cuando el usuario hace clic o escribe.

Modificar contenido

textContent

- Obtiene o establece el contenido textual de un nodo y de todos sus descendientes de forma **literal y directa** desde el DOM

innerText

- Obtiene o establece el texto tal como es **renderizado y visible** para el usuario en la pantalla.

Selección de Nodos en el DOM



querySelector

Selecciona el **primer elemento** que coincida con el selector CSS especificador.

```
const btn =  
document.querySelector('.btn-  
primary');
```



querySelectorAll

Retorna una colección (**NodeList**) con todos los elementos coincidentes.

```
const items =  
document.querySelectorAll('li.item');
```



getElementById

Método directo y rápido para seleccionar un elemento por su identificador único **ID**.

```
const form =  
document.getElementById('main-  
form');
```

Creación e Inserción de nodo



createElement()

Instancia un elemento en memoria fuera del árbol activo: **const card = document.createElement("div").**



append() (Moderno)

Permite insertar nodos y texto directo sin necesidad de **createTextNode:**
contenedor.append(card).

Eliminación de Nodo

Método Moderno: `nodo.remove()`

Elimina el elemento del DOM directamente sin necesidad de llamar al nodo padre:

```
const item =  
document.querySelector("#card");  
item.remove(); // Directo y limpio
```

Método Legacy: `padre.removeChild(hijo)`

Uso tradicional / legacy: Requiere invocar al nodo padre contenedor para desacoplar a un hijo específico.

```
const padre = item.parentNode;  
padre.removeChild(item); // Enfoque  
antiguo
```

Eventos

Un **evento** en JavaScript es una acción o suceso detectado por el navegador —provocado por el usuario o por el propio sistema— ante el cual podemos programar una respuesta inmediata.

Eventos comunes:



Click: Se dispara cuando el usuario hace clic en un elemento.



Input: Se activa cuando cambia el valor de un campo de entrada.



Submit: Se dispara cuando el usuario envía un formulario.



mouseover / mouseout: Se activan cuando el cursor entra o sale de un elemento.

Manejar Eventos desde HTML

Manejar eventos desde HTML consiste en vincular la ejecución de un script o una función de JavaScript directamente dentro de los atributos de una etiqueta HTML.

```
9      <!-- HTML: El evento 'onclick' llama a la función de JS -->
10     <button onclick="mostrarSaludo()">Haz clic aquí</button>
11     <p id="mensaje"></p>
12
13     <script>
14         // JavaScript: Función que se ejecuta al hacer clic
15         function mostrarSaludo() {
16             const texto = document.getElementById("mensaje");
17             texto.innerText = "¡Hola! Ejecutaste un evento desde HTML.";
18         }
19     </script>
```

Haz clic aquí

¡Hola! Ejecutaste un evento desde HTML..

Manejar Eventos desde JS

El método **addEventListener()** permite registrar un "escuchador" de eventos en un elemento HTML específico para ejecutar una función cuando dicha acción ocurre, manteniendo el comportamiento en JavaScript totalmente separado de la estructura HTML.

elemento.addEventListener("tipoDeEvento", funcionAEjecutar);

donde :

- **elemento:** El nodo del DOM seleccionado al que se le aplicará la acción.
- **tipoDeEvento:** String con el nombre del evento **sin** el prefijo *on* (ej: 'click', 'submit', 'keydown').
- **funcionAEjecutar:** La función (callback) con las instrucciones que se correrán cuando el evento ocurra.

Manejar Eventos desde JS

```
<button id="btn-cambiar">Cambiar Color</button>
<div id="caja">Caja de prueba</div>

<script src="evento.js"></script>
```

```
1 // 1. Seleccionar los elementos
2 const boton = document.getElementById("btn-cambiar");
3 const caja = document.getElementById("caja");
4 // 2. Definir la función de respuesta
5 function cambiarFondo() {
6     caja.style.backgroundColor = "lightblue";
7 }
8 // 3. Escuchar el evento 'click'
9 boton.addEventListener("click", cambiarFondo);
```

Buenas Prácticas

- **Separación de Responsabilidades:** HTML define la estructura y el contenido; JavaScript controla el comportamiento y la lógica.
- **Mantenibilidad:** Editar un único archivo **.js** es más rápido que modificar atributos **onclick** repartidos en decenas de etiquetas HTML.
- **Múltiples Escuchadores:** **addEventListener()** permite adjuntar varios eventos o funciones al mismo elemento sin sobrescribir el código anterior.

El Objeto Evento (event /e)

Concepto: Cada vez que se dispara un evento, el navegador crea automáticamente un objeto con información detallada sobre la acción ocurrida.

Propiedades Principales:

- **event.target:** Referencia directa al elemento HTML que provocó el evento.
- **event.type:** Nombre del evento ejecutado (ejemplo: 'click', 'submit').
- **event.key:** Devuelve la tecla exacta presionada en eventos de teclado.

```
1  const boton = document.getElementById("btn-guardar");
2  boton.addEventListener("click", function(event) {
3    console.log("Elemento cliqueado:", event.target);
4  });
```

Captura de Datos (`input.value`)

Concepto: Para obtener el texto o valor ingresado por el usuario en campos de formulario (`<input>`, `<select>`, `<textarea>`), se utiliza la propiedad `.value`.

Diferencia Clave: Los elementos de entrada no usan `innerText` ni `textContent`; su contenido se almacena siempre en `.value`

```
1  const inputEmail = document.getElementById("user-email");
2  const boton = document.getElementById("btn-obtener");
3
4  boton.addEventListener("click", function() {
5    // Capturamos el texto del input al momento de hacer clic
6    const emailIngresado = inputEmail.value;
7    console.log("El correo ingresado es: " + emailIngresado);
8
9  });
```

Evento Submit y event.preventDefault

- **Comportamiento por Defecto:** Al hacer clic en un botón de envío o presionar Enter dentro de un `<form>`, el navegador recarga la página automáticamente.
- **event.preventDefault():** Método que detiene el comportamiento por defecto de un evento, permitiendo controlar la lógica con JavaScript sin perder los datos ni refrescar la pantalla.

```
1  const formulario = document.getElementById("mi-form");
2  formulario.addEventListener("submit", function(e) {
3      e.preventDefault(); // Cancela la recarga automática de la página
4      console.log("Formulario capturado por JS");
5
6  });
```

Validación de Formulario

Objetivo: Verificar que los datos ingresados cumplan con las reglas requeridas antes de procesarlos.

Uso de `.trim()`: Elimina espacios en blanco al inicio y al final para evitar entradas vacías compuestas solo por espacios.

```
1  const form = document.getElementById("form-registro");
2  const inputNombre = document.getElementById("nombre");
3  const mensaje = document.getElementById("mensaje-error");
4  form.addEventListener("submit", function(e) {
5      e.preventDefault();
6      if (inputNombre.value.trim() === "") {
7          mensaje.textContent = "El nombre es obligatorio.";
8          mensaje.style.color = "red";
9      } else {
10         mensaje.textContent = "¡Registro exitoso!";
11         mensaje.style.color = "green";
12     }
13
14 });
```